ReadWise

Module 1 — Project Overview & Architecture

---

1. Simple Explanation (ELI5)

Imagine you're reading a hard textbook chapter on your laptop. Every few sentences, you hit a word you don't know, or a concept you want explained better. Normally you'd:

- • Open a new tab
- • Google the word
- • Maybe open ChatGPT in another tab
- • Copy-paste the confusing paragraph
- • Read the explanation
- • Try to remember the word to review later
- • Switch back to your PDF

That's **7 context switches** just to understand one sentence. ReadWise's whole reason for existing is to collapse all of that into **one screen**: you highlight text, and right there — no new tabs — you get a dictionary definition, an AI explanation, and the option to save the word for later practice (like flashcards).

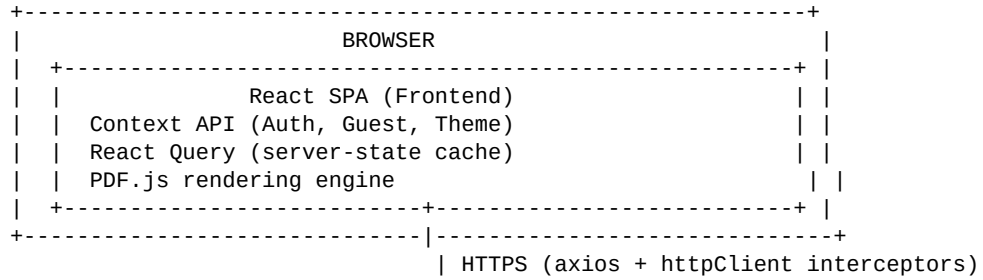
Think of it like **Kindle + ChatGPT + Anki, glued together, focused specifically on PDFs.**

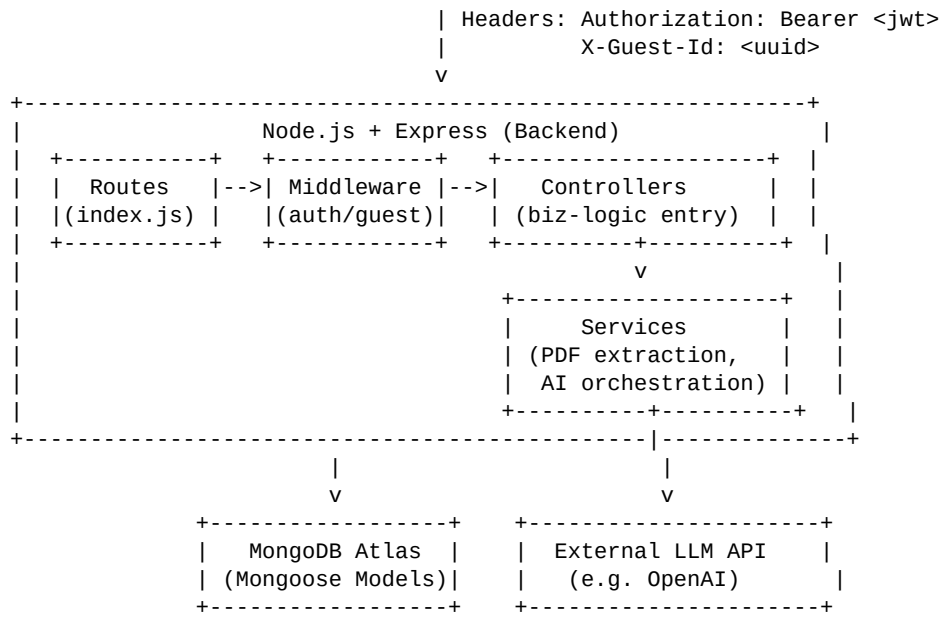
2. Technical Explanation

ReadWise is a **full-stack, decoupled web application**:

- • **Frontend:** React SPA (Single Page Application) that renders PDFs client-side, manages UI state via Context API, and manages server state via React Query.
- • **Backend:** Node.js + Express REST API that handles authentication, file storage, PDF text extraction, AI orchestration, and CRUD for all user data.
- • **Database:** MongoDB (via Mongoose ODM) — a document database, which fits well here because the data (highlights, notes, chat messages) is naturally nested/flexible rather than strictly tabular.
- • **Two identity systems coexist:** authenticated users (JWT) and anonymous guests (a UUID stored in localStorage, sent via a custom header). This is the freemium engine of the app.
- • **AI is not embedded in the frontend** — it's proxied through the backend, which calls an external LLM API (like OpenAI). The frontend never talks to the LLM directly.

Here's the 30,000-foot architecture diagram:





3. Why This Architecture Was Chosen

Let's reason through each major decision like the engineer who made it would have to defend it in a design review.

Why a decoupled client-server REST architecture (not server-rendered, not monolith-with-templates)?

A PDF reader is **highly interactive** — zooming, highlighting, dragging selection toolbars, live AI streaming. That kind of rich client-side interactivity is exactly what SPAs are built for. Server-rendered HTML (like traditional Express + EJS/Pug) would mean full page reloads or heavy AJAX patching for every interaction — painful for this UX.

Why MongoDB instead of a relational DB (PostgreSQL/MySQL)?

Look at the shape of the data: a **Highlight** might have `{ page, coordinates: [{x,y}], color, text }`. A **PDFPage** might have `structuredContent` — potentially nested text blocks. This is **document-shaped data**, not strictly tabular. MongoDB lets you store this naturally without needing a dozen joined tables. Also, early-stage products iterate on schema fast — Mongoose's flexible schema is cheaper to evolve than relational migrations during MVP stages.

Why React Query for server state instead of just Context/Redux?

Server data (PDFs list, highlights, vocabulary) has fundamentally different needs than local UI state:

- • It can go stale (someone uploads a PDF elsewhere)
- • It needs caching to avoid refetching on every render
- • It needs loading/error states
- • It benefits from automatic revalidation

React Query gives you caching, deduplication, background refetching, and invalidation **for free**, instead of hand-rolling `useEffect` + `useState` fetch logic everywhere (which is what teams did before React Query existed, and it was a bug factory).

Why dual identity (JWT + Guest UUID) instead of forcing signup immediately?

This is a **product/growth decision**, not just a technical one. Forcing signup before someone can try the product kills conversion — most users won't sign up for something they haven't experienced yet. So ReadWise lets you use the app anonymously (tracked by a `guestId` in localStorage) with limited usage, and only asks you to sign up once you've hit value (like the free tier of Notion, Figma, Canva). We'll go very deep on this in the Freemium module.

Why proxy AI calls through the backend instead of calling OpenAI directly from React?

Three reasons, and this is a classic interview question:

- • **Security** — your OpenAI API key would be exposed in browser JS if called client-side. Anyone could open devtools and steal it.
- • **Context injection** — the backend needs to fetch the relevant PDFPage text from MongoDB and inject it into the AI prompt. The frontend doesn't have direct DB access.
- • **Rate limiting / cost control** — the backend can enforce guest limits (quickExplainUsed, deepExplainUsed) before ever spending money calling the LLM.

4. Alternative Approaches (and Why They Weren't Chosen)

Alternative	Why it wasn't chosen
Next.js (SSR/SSG)	PDF reading is a fully client-side, session-based experience — SSR buys little here, adds complexity
Redux/Zustand for everything	React Query already solves server-state; Context is enough for the small amount of pure UI state (theme, auth session)
PostgreSQL	Would work, but requires more upfront schema rigidity for data that's naturally nested (highlights, chat threads)
Firebase/Supabase (BaaS)	Faster to bootstrap, but less control over custom business logic like PDF text extraction pipelines and AI orchestration
Force signup before any usage	Kills top-of-funnel conversion; freemium/guest-first is standard practice for consumer SaaS
Client-side direct OpenAI calls	Exposes API key, no server-side rate limiting, no DB context injection

5. Pros & Cons of This Architecture

Pros:

- • Clean separation of concerns (frontend/backend can scale and deploy independently)
- • MongoDB's schema flexibility speeds up feature iteration
- • React Query eliminates a huge class of stale-data/loading-state bugs
- • Guest-first flow maximizes trial conversion without backend complexity of "real" anonymous accounts
- • Backend-proxied AI calls are secure and controllable

Cons:

- • MongoDB's flexibility can lead to inconsistent document shapes over time if discipline isn't enforced (no foreign key constraints, no schema migrations by default)
- • Dual identity systems (JWT + Guest UUID) add real complexity — every protected route needs to handle two different auth paths, and the migration flow (guest to real account) is a common source of bugs
- • No mention (yet) of a caching/CDN layer for PDF files — could become a bottleneck at scale
- • Local file storage (/uploads folder) means the backend isn't horizontally scalable without shared storage (S3, etc.) — a single server instance would need to own all uploaded files

6. Real-World Example (Full Journey)

Let's trace one full user story through the entire stack, since this is the best way to actually internalize how the modules connect:

"Priya, a college student, visits ReadWise for the first time to study a chapter of her physics textbook PDF."

1. Priya opens the site → App.js mounts → GuestSessionProvider checks localStorage for rw_guest_id → none found → generates a new UUID → stores it in localStorage.
2. She uploads her PDF via Uploadcard.jsx → this fires a multipart/form-data POST to /api/pdfs/upload, tagged with her X-Guest-Id header (not a JWT, since she's not logged in).
3. Backend's pdfLibraryController.js receives the file, saves it physically to /uploads, creates a PDF document in MongoDB tagged with her guestId (not userId, since she has none yet), and kicks off text extraction into PDFPage documents (one per page, storing raw text for later AI/search use).
4. She's redirected into ReaderLayout.jsx, which loads PDFViewer.jsx (renders the actual PDF using PDF.js) and ReaderSidebar.jsx.
5. She highlights a confusing sentence about quantum tunneling and clicks "Quick Explain" in SelectionToolbar.jsx.
6. This calls useAiChat.explainSelection() → POST /api/ai/explain with her selected text + X-Guest-Id.
7. Backend middleware guestAuth.js checks: has she used up her free quick-explain quota (stored in GuestUsage.quickExplainUsed)? She hasn't — this is her first one — so it's allowed and the counter increments.
8. explainController.js fetches the relevant PDFPage.text as context, builds a prompt, and calls the external LLM API.
9. The explanation streams back and renders in AiPanel.jsx.
10. She keeps reading and hits her limit (say, 3 free quick-explains). On the 4th attempt, guestLimiter middleware rejects the request with a 403 → the frontend's axios interceptor catches this globally → PremiumModal.jsx pops up.
11. She decides to sign up. AuthProvider.jsx handles her signup → backend creates a real User document and returns a JWT.
12. Immediately after, the frontend calls migrateToAccount(token) → POST /api/auth/migrate sending her guestId → backend reassigns every document (PDF, Highlight, GuestUsage stats, etc.) that had her guestId to now belong to her new userId.
13. Her localStorage guest data is cleared, and from now on all requests carry Authorization: Bearer <jwt> instead of X-Guest-Id.

This single story touches almost every module in the Feature Inventory — which is exactly why understanding this flow first makes every individual feature module make sense later.

7. Interview Section

Q: "Walk me through the architecture of a project you've built."

How to answer: Don't recite folder names. Say it like this: "It's a decoupled React/Express app. The frontend handles rendering and interactivity, React Query manages server-state caching, and the backend is a REST API backed by MongoDB. The two things I'd highlight as interesting engineering decisions are: (1) a dual-identity auth system supporting both guests and registered users for freemium conversion, and (2) proxying all LLM calls through the backend for security and context-injection reasons." This shows you understand why, not just what.

Q: "Why MongoDB over a relational database here?"

How to answer: Expected answer: talk about the document-shaped nature of the data (highlights, nested chat messages) and schema flexibility during early iteration — not "because it's popular" or "because it's easy." Interviewers want to hear trade-off reasoning, not buzzwords.

Q: "Why not just call OpenAI from the frontend?"

How to answer: Expected answer: API key security, backend-enforced rate limiting, and the need to inject database context (extracted PDF text) that only the backend has access to. This is one of the most common "gotcha" questions for AI-integrated apps — interviewers are checking if you understand basic API-key security.

Q: "What's a weakness in this architecture?"

How to answer: Good answers show self-awareness: local file storage for PDFs won't scale horizontally; MongoDB's schema flexibility needs enforced discipline (via Mongoose schemas + validation) or you get inconsistent documents over time; dual-identity auth adds middleware complexity on every protected route.

What interviewers actually expect from this kind of question: they're not testing whether you memorized file names. They're testing whether you can reason about **trade-offs** — every technology choice has a cost, and they want to see you can articulate what was gained and given up.

8. Common Mistakes (Junior-Level Pitfalls in This Kind of Architecture)

- • Forgetting to handle both auth states (JWT and Guest) in a new protected route — easy to test only as a logged-in user and ship a bug for guests.
- • Treating MongoDB like a relational DB — writing code that assumes referential integrity (e.g., assuming a pdfId on a Highlight always points to an existing PDF) without validating it, since Mongo won't enforce foreign keys for you.
- • Calling the LLM API without a context window strategy — if PDFPage.text is large, blindly stuffing all pages into the prompt burns tokens and money fast.
- • Not invalidating React Query cache after mutations (e.g., creating a highlight) — leads to stale UI where the highlight doesn't appear until a manual refresh.

9. Best Practices Reflected Here

- • Feature-based frontend folder structure (features/ai, features/vocabulary, etc.) instead of type-based (components/, reducers/, hooks/ all flat) — this scales much better as the app grows, because everything related to one feature lives together.
- • Middleware-based cross-cutting concerns (guestAuth.js, auth.js) instead of duplicating auth checks inside every controller.
- • Caching expensive external calls (DictionaryCache model) to avoid redundant third-party API costs.
- • Separating Controllers (HTTP concerns) from Services (business logic) — this makes business logic testable without spinning up Express.

10. Senior Engineer Review — Production Improvements

If I were the lead engineer reviewing this architecture for scaling to production traffic, here's what I'd flag:

1. Move file storage off local disk → to S3/Cloudflare R2. Local /uploads breaks horizontal scaling (multiple server instances can't share a disk) and has no redundancy.
2. Add a schema validation layer (Zod/Joi) at the controller boundary — MongoDB won't stop malformed documents from being written; validate at the edge.
3. Add API-level rate limiting beyond just guest quotas — protect against abuse/DDoS on /auth/login and AI endpoints specifically, since those are the most expensive to abuse.
4. Stream AI responses (Server-Sent Events or WebSockets) instead of waiting for full completion — massively improves perceived latency for chat/explain features.
5. Add structured logging + monitoring (e.g., Sentry, Datadog) around the AI orchestration layer specifically, since LLM API failures/timeouts are the most likely production incident source.
6. Background job queue for PDF text extraction (e.g., BullMQ + Redis) instead of doing it synchronously during upload — large PDFs would otherwise block the request/response cycle.

End of Module 1. Next: PDF Library & Freemium Authentication.